INSTITUTO TECNOLÓGICO DE BUENOS AIRES

INTERPRETACIÓN SEMÁNTICA DE SOLICITUDES WEB EN CAPA DE APLICACIÓN

Alumno: Cozzi, Alejandro Luis (Leg. N° 27826374)

PROFESOR: Salgado, Ariel

TALLER: Introducción a la Metodología de Investigación – Maestría en Ciencia de Datos

BUENOS AIRES
SEGUNDO CUATRIMESTRE - 2026

1. Introducción

1.1. Contexto

En el cine de acción, es habitual ver personajes asociados al ciberdelito, utilizados para resolver situaciones que demanda la trama. Eso en parte se debe a que en las últimas décadas, muchos de los activos más valiosos del planeta son digitales, como lo podría ser un tren o un banco a mediados del siglo pasado. Pero esa imagen que ha construido el cine, de un personaje aislado escribiendo cosas a gran velocidad y penetrando sistemas, tiene poca relación con la realidad, que habitualmente suele ser menos atractiva y mucho más cercana. Un ejemplo real son los ataques de ingeniería social, orientados a tomar el control del *Whatsapp* de la víctima, o incluso de su Home Banking, ambos son parte del mismo problema, así como el correo del príncipe Nigeriano[1] que nos contacta para sacar fortunas de su país, el ícono del Messenger que se tiene que poner azul si reenviamos un mensaje a todos nuestros contactos (bajo amenaza de la eliminación de la cuenta) o los millones de mensajes que llegan a diario con felicitaciones por haber ganado un premio.

Del otro lado, los proveedores de servicios intentan concientizar a los usuarios, alertando sobre estafas, reenviando a la carpeta de Spam de los correos sospechosos o bloqueando la clave cuando alguien se equivoca varias veces, por tratarse de un posible ataque de fuerza bruta. La seguridad se asume como responsabilidad compartida y se distribuye en todos los niveles posibles, desde el desarrollo seguro de aplicaciones, filtros en la red perimetral, doble factor de autenticación, etc.

En uno de esos niveles se encuentra el cortafuegos de aplicación o WAF (Web Application Firewall), una tecnología desarrollada en la década de 1990 (vinculada al nacimiento de la WWW en 1993) especializada en analizar el tráfico web que llega a una aplicación y tomar acciones sobre esas solicitudes, en definitiva bloquearlas o dejarlas pasar.

Los WAF tradicionalmente aplican fórmulas de expresiones regulares (de ahora en más regex), basadas en firmas de ataques conocidos, sobre los requerimientos del cliente, aunque en los últimos años han sumado técnicas más variadas como listas de reputación de ips, bloqueos por geolocalización o detección de bots en base al comportamiento. Este trabajo apunta centralmente a la mejora de ese filtro basado en firmas de ataques conocidos.

Los CVE (Por *Common Vulnerabilities and Exposures*) son esos **ataques conocidos** mencionados, cada uno de ellos tiene un código de registro utilizado para realizar su seguimiento, el software afectado, formas de mitigación y relación con otros CVEs. A su vez, añade una capa de trazabilidad por lo que cada CVE queda ligado a la publicación de los parches y nuevas reglas de WAF cada vez que surge uno nuevo. En el mismo nombre, radica su debilidad: se trata de reglas y parches relacionados a vulnerabilidades conocidas, asumiendo de facto que antes de su publicación eran desconocidas.

1.2. Justificación del estudio

Imaginemos un escenario donde un ciberdelincuente logra una forma de penetrar un sistema a partir de una vulnerabilidad del mismo (desconocida hasta el momento). En un lapso de tiempo (que podría durar entre un segundo y diez años) el responsable lo detecta y lo reporta. En ciberseguridad, ese momento se llama ***zero-day***, aun cuando la vulnerabilidad pueda haber existido desde tiempo atrás, se trata del día cero para el camino de soluciones posibles a ese problema.

El reporte de esa vulnerabilidad nueva, sería resuelto por una comunidad de colaboradores o por equipos de desarrollo, luego sería liberado el parche y posiblemente una regla de WAF asociada a esa vulnerabilidad. Después, los responsables de otros servicios con el mismo software en todo el mundo, aplicarían la actualización y el sistema volvería a ser seguro.

En un mundo ideal, ese proceso completo duraría un par de semanas, en el mundo real de 2025 ese proceso duró una media de 241 días, según el reporte de Cost of Breach que IBM publica cada año[1].

[1] `https://www.avast.com/c-nigerian-prince-scam`

Si bien como demuestra el informe, ese proceso ha mejorado en los últimos años (en parte gracias a los paradigmas de desarrollo ágil y filosofías de CICD) es probable que nunca sea suficiente para cerrar esa ventana temporal de exposición, siguiendo ese pipeline de detección-reporte-actualización.

Este trabajo busca explorar posibles soluciones o reducciones de ese gap, a partir de técnicas y algoritmos de aprendizaje automático, aplicadas en la capa de WAF para detectar solicitudes maliciosas.

Un ejemplo no imaginario fue el caso de un CVE conocido como «Log4Shell» de 2021 reportado por un empleado de *Ali Baba Cloud* el 24 de noviembre y asumido por *Apache Software Foundation* el 10 de diciembre de ese mismo año. *Akamai*, un productor privado de WAF, publicó parches al día siguiente [2] y el Instituto Nacional de Estándares y Tecnología (de EEUU, NIST por sus siglas en inglés) en [3] el 12 de diciembre del mismo año. Como se señala en [4] la vulnerabilidad existía al menos desde 2016, cuando se publicó el trabajo de [5] y cuya existencia podría remontarse hasta 2013, año en que se publicó el *commit* que dio origen a la vulnerabilidad [6]. Los primeros parches no fueron suficientes, lo que requirió nuevas versiones publicadas sobre el mismo, hasta el 28 de diciembre de 2021 [7].

La vulnerabilidad tuvo notoriedad mundial ya que afectó a plataformas populares como Minecraft, Twitter y Cisco, y su difusión permitió miles de ataques efectivos en los primeros días y cientos de millones de dispositivos afectados [8]. Es difícil establecer una ventana temporal de exposición global para «Log4Shell», pero podríamos decir que como mínimo fue de alrededor de un mes, y como máximo, de 5 años.

Las técnicas aplicadas en «Log4Shell» no diferían demasiado de los ataques conocidos previamente. Sobre este supuesto, tiene sentido la propuesta de pensar un sistema de aprendizaje automático que trabaje junto a las reglas de WAF pero de forma más flexible que las mismas, basados en el comportamiento continuo de las solicitudes.

1.3. Alcances del trabajo y limitaciones

Clasificar solicitudes web utilizando técnicas y algoritmos de aprendizaje automático, trabajando junto al WAF, de mínima podría mejorar los ratios de detección e incluso detectar reglas que no se han configurado o no se han actualizado correctamente. De máxima, podría detectar una vulnerabilidad de ***zero-day***, antes de la publicación por los organismos de seguimiento y parcheo. No obstante es necesario tener en consideración las limitaciones intrínsecas a la propia propuesta desde lo práctico y lo conceptual.

En principio, el set de datos utilizado, proviene de un entorno controlado [9], desplegado *ad hoc* para investigación y no de un entorno de producción real. Los logs de tráfico web contienen información sensible, valiosa y confidencial que otorga demasiada información sobre el negocio y el desarrollo, por lo cual es esperable que en general no se publiquen.

Por otro lado, como se explica en el punto 3, actualmente se reconocen cerca de 80 tipos de ataques y es poco probable que un dataset contenga a todos los tipos de ataques y en cantidades medianamente balanceadas. No está de más mencionarlo aunque parezca trivial: los tipos de ataques que no estén en el entrenamiento, tampoco estarán en la predicción.

Si el insumo de datos principal es un conjunto de datos etiquetados, el proyecto nace con una dependencia externa, que a su vez plantea otras dificultades: el WAF etiquetando produce falsos positivos, los cuales serían tomados como referencia, contaminando el entrenamiento del modelo. Si en su defecto quisiéramos realizar el etiquetado a partir de análisis forense humano, el tiempo y costo de la producción de datos se dispararía, imposibilitando procesos de reentrenamiento automatizado y aun así, sin garantizar la certeza de las etiquetas.

En cualquier caso la solución no se propone como un reemplazo al WAF y sus reglas, sino como un complemento adyacente al WAF que pueda ofrecer una solución híbrida para aumentar la seguridad. Quedaría pendiente de analisis la forma técnica en que esa solución podría integrarse de forma eficiente en un WAF para trabajar en conjunto, sin quitarle rendimiento al tráfico.

2. Estado de la cuestión

Mejorar los niveles de detección de ataques en WAF a partir de los datos, es un problema constante y recurrente dado que la cantidad y creatividad de los ataques se sostiene en el tiempo. Las dinámicas elementales desde el punto de vista técnico para quienes no estén familiarizados con el problema de la seguridad y la acción del WAF son trabajadas en [10] donde se realiza una exploración metodológica acerca de la arquitectura necesaria para detectar y bloquear amenazas de tipo SQLI; si bien menciona los aportes que podrían hacer las técnicas de deep learning en esa capa, no incluye ninguna en su propuesta. Sí aporta algunas nociones metodológicas que podrían ayudar en la etapa de experimentación y, sobre todo, al momento de comparar con los filtros heurísticos.

En cambio, [11] realiza una aproximación muy similar a la que pretende plantear este trabajo, aunque sobre presupuestos metodológicos diferentes. Si bien parte de arquitecturas transformers para la comprensión semántica de cada solicitud web, apunta al fine-tuning a partir de RoBERTa, antes que a la extracción de embeddings crudos para su posterior procesamiento. No buscan generalizar sobre un conjunto de solicitudes maliciosas per se, sino que proponen realizar un entrenamiento de modelos singulares para cada aplicación web.

Es probable que el entrenamiento singular tenga mucho sentido a la hora de detectar anomalías, aunque no queda del todo claro en ese trabajo cómo se realizaría el etiquetado ya que en los experimentos se utilizan datasets ya etiquetados como CSIC 2010 y DRUPAL.

En una segunda etapa, luego del fine-tuning de RoBERTa, utilizan OneClassSVM un clasificador binario para detectar los ataques en los datasets, y lo comparan con los aciertos(Ratio de aciertos o True Positive Rate, de ahora en más TPR) y falsos positivos (en adelante FPR por False Positive Ratio) usando ModSec como WAF.

La potencia de RoBERTa también fue utilizada en [12] donde los autores experimentaron con un ensamble bastante más complejo pero con algunos parámetros similares. Su trabajo también utiliza herramientas transformers, RoBERTa en particular y el dataset de CSIC 2010 para pruebas, además del dataset ATRDF 2023. Su propuesta se inspira en el framework GAN de forma muy original: identifican tokens reservados en el dataset original y los enmascaran con un token de máscara (literalment <MASK>), luego utilizan RoBERTa para generar el reemplazo esperable de esa fracción del código, lo cual entrega un dataset paralelo, distinto del original. A continuación utilizan nuevamente RoBERTa para comparar ambos, y calculan la similaridad del coseno de cada palabra para determinar anomalías. Por último, entrenan un clasificador de Random Forest para predecir y discriminar entre «tráfico normal» y «posible ataque».

Otra característica en común de los dos trabajos mencionados anteriormente es que ambos obtienen como resultado esa clasificación binaria.

En cambio en [13] los autores apuntan a entregar ambos tipos de clasificación, una binaria (que discrimine ataque / legítimo) y una multiclase, clasificando hasta 8 clases en total (incluyendo al tráfico legítimo). Partiendo de la extracción de embeddings de la solicitud http, proceden a añadirle ruido usando el algoritmo Da-Sana. Para construir los embeddings utilizaron Keras con un padding posterior para normalizar los largos (es decir recortando las líneas que exceden el largo determinado y completando con ceros las que no llegan a él) usando los datasets CSIC-2010 y ECML/PKDD.

Un desarrollo más amplio y complejo se encuentra en [14]. En este trabajo se presenta un proceso que acuña su propio nombre: DeepPTSD una sigla que describe básicamente el proceso propuesto, de Deep Learning Payload Traffic Detection Method Transfer Semi-Supervised Detection. Se trata de un ensamble bastante sofisticado que se entrena con datasets clasificados públicos y aspira a detectar líneas maliciosas en un WAF. A grandes rasgos el ensamble sigue un proceso similar a los demás, aunque con variantes en cada paso, por ejemplo los embeddings se extraen con Word2Vec luego se pre-entrena un modelo con esa información usando redes convolucionales. Con los resultados se realiza una aumentación de los datos que sigue dos caminos: «keyword mining» que parte de un dataset de payloads y aplica una ecuación tf-idf para crear una biblioteca de payloads y luego una «payload perturbation» que reemplaza ciertos tokens que

detecta similares a los de la biblioteca en la línea a analizar.

Los trabajos mencionados parecen coincidir en un problema central: las reglas de WAF son eficientes para detectar un alto porcentaje de solicitudes maliciosas, pero son incapaces de detectar algunos tipos de ofuscación. Además, por su propia naturaleza, surgen como reacción a la detección de vulnerabilidades de tipo ***zero-day*** y esa reacción abre una ventana de oportunidad para el ciberdelito.

Es por eso que se busca en todos una solución al problema a partir de los datos, con distintas estrategias que en definitiva resultan en una acción inmediata sobre una solicitud web, en la capa de WAF. Esta solución no implica un reemplazo del sistema de reglas tradicional, sino un complemento que mejore las métricas de detección. En cualquier caso, eso significa básicamente aumentar el TPR y reducir el FPR con distintas estrategias específicas de la ciencia de datos, en un margen de tiempo poco considerable.

Ninguno de los trabajos mencionados probó el rendimiento comparado de distintos modelos transformers, ni su combinación con distintos algoritmos de clasificación. Pero más allá de del aporte cuantitativo que podría aportar el testeo de combinaciones, no se está teniendo en cuenta el gran peso que podría tener un conjunto de variables obtenidas a partir de ingeniería de características, sobre los rendimientos en un modelo predictivo dentro de un WAF.

3. Definición del problema

La seguridad de los activos digitales es un problema a nivel global que dista de poder resolverse de forma taxativa. Hay muchos enfoques al respecto, en este trabajo se toma como referencia el modelo OSI [15] definido por la organización internacional ISO para la seguridad en las comunicaciones computacionales. Más allá del problema de seguridad global, este trabajo se focalizará en tres problemas dependientes de este, los cuales se intentarán explicar a continuación.

El modelo OSI define 7 capas abstractas de seguridad, una de las cuales, la capa 3 se ocupa de la seguridad de red. Es en esa capa donde se colocan los cortafuegos de aplicación, un filtro lógico que media entre una aplicación e internet y se ocupa, entre otras funciones, de definir si una solicitud es legítima o no para proceder a su bloqueo o redireccionamiento hacia el servicio.

Esa definición no es sencilla por varios motivos y la definición que se toma no siempre es la correcta. La OWASP Foundation (por sus siglas en inglés Open Web Application Security Project) reconoce 78 distintos tipos de ataques, aunque la cifra está en constante movimiento. Cada uno de ellos no llega al WAF con una etiqueta de «malicioso» en el encabezado, sino que lo habitual es que se recurra a distintas técnicas de ofuscación para evitar la detección, lo cual constituye el primer problema.

Y esa detección, recurre principalmente a expresiones regulares que buscan patrones dentro de la solicitud que podrían ser signo de un ataque de los mencionados.

OWASP provee de forma abierta cerca de 300 reglas para ese fin a través del programa CRS (Owasp Core Rule Set) que se pueden implementar en cualquier WAF. Owasp desarrolló un WAF de código abierto, llamado ModSecurity (de ahora en más Modsec) al cual se le pueden definir esas reglas, con distintos niveles de paranoia. A mayores niveles de paranoia, más cantidad de falsos positivos, a menor, más posibilidades de bypass[2], lo cual constituye el segundo problema.

El tercer problema es procedimental: Supongamos que un atacante descubre una vulnerabilidad en un sistema (zero-day), los responsables de ese servicio vulnerado detectan la intrusión, lo reportan, Owasp u otro organismo produce una regla para ese exploit, publican la regla y se reconfigura el WAF con la actualización. Este pipeline produce necesariamente una ventana de oportunidad muy importante como se mencionó en el punto 1.2.

[2]La misma expresión que se utiliza en medicina, en seguridad se refiere a saltear una regla o un filtro.

4. Hipótesis

A partir de un conjunto de líneas de log de WAF con tráfico web, etiquetados con una clasificación de ciberseguridad (normal o malicioso, especificando el tipo de ataque) es posible generar nuevas etiquetas para líneas desconocidas, sin necesidad del filtro heurístico del WAF sino con un algoritmo que determine la naturaleza de la solicitud, realizando un trabajo igual o superior al que realiza un WAF.

5. Objetivos

5.1. Objetivo General

Clasificar correctamente un conjunto de líneas de log asociadas a tráfico web normal o malicioso, con un ratio igual o superior al que realiza actualmente un WAF.

5.2. Objetivos Específicos

- Extraer de cada línea de log, las características semánticas particulares que nos aporten información sobre la intencionalidad de la solicitud.
- Identificar las características de la línea que más peso tienen dentro de la definición de la etiqueta (mediante feature engineering).
- Entrenar un modelo clasificador que identifique el tipo concreto de ataque en la solicitud (o la ausencia del mismo).
- Generar un pipeline eficiente que permita procesar solicitudes en menos de 3 segundos.

6. Solución Propuesta

Propongo desarrollar un modelo de aprendizaje automático supervisado que logre interpretar semánticamente las solicitudes web que llegan a un WAF y, en base a esa interpretación, clasificarlas entre los distintos tipos de ataques posibles o solicitud legítima.

Para ello, partiendo de un set de datos previamente clasificado, extraer embeddings de ciertos campos a partir de modelos transformers, extraer características con distintas técnicas de ingeniería y con esa información, entrenar un modelo con un algoritmo clasificador, que pueda hacer esa clasificación sobre datos sin clasificar.

La falta de especificidad del modelo transformers y el algoritmo clasificador se debe a que probaré con distintas combinaciones de ensamble para evaluar los rendimientos de cada combinación.

7. Metodología

Se espera lograr un pipeline de datos que permita la experimentación y prueba de distintos modelos y librerías. Para el entorno global se utilizará Kubernetes como infraestructura de soporte, que provea contenedores necesarios para las tareas basados en la filosofía DevSecOps.

a) Ingesta

El software de soporte inicial sería una base de datos Postgres (dentro de un contenedor) que podría usar psql para ingestar u otro contenedor con Python y SQLAlchemy para la interacción.

Por la misma naturaleza de las líneas de log (muchas con caracteres especiales y binarios) es necesario insertar línea por línea con una codificación amplia como «latin-1» con un control flexible de errores. Ello no implica descartar ninguna línea (ya que justamente un outlier en este caso podría ser la clave de un ataque) sino por el contrario, garantizando que todas las líneas lleguen a la tabla.

El producto de esta primer etapa debería ser una tabla Postgres que contenga un id (construido a partir de un hash md5 de cada línea), la línea en cuestión como string y su/s label/s como string o vector de strings.

b) Limpieza y parseo

Esta etapa es compleja por varios motivos y las decisiones que se tomen en ésta pueden cambiar de raíz los resultados finales. En principio no se puede realizar una normalización como se haría con otros sets de datos, ya que eliminar caracteres especiales o aplanar el case (llevar todo a mayúsculas o a minúsculas), nos haría perder información elemental para nuestro objetivo.

Por otro lado, la presencia, dentro del propio contenido de los datos, de símbolos que habitualmente se emplean como separadores de campos incrementa la complejidad del proceso de parseo. Caracteres como la coma (`,`), el punto y coma (`;`), el numeral (`#`) o incluso caracteres de control como el salto de línea (`\n`), el retorno de carro (`\r`) y la tabulación (`\t`) son utilizados con frecuencia por los atacantes para alterar el comportamiento esperado de los mecanismos de procesamiento de datos. Incluso una solicitud legítima puede contener comas (`,`) y eso rompería la estructura de un CSV.

En consecuencia, es necesario analizar individualmente cada conjunto de datos para identificar con precisión la posición de cada variable y asignarla correctamente a los campos de la estructura tabular que serán utilizados en las etapas posteriores del procesamiento.

c) Selección de variables e ingeniería de características

Se debería realizar un concatenado del método más el cuerpo de la solicitud (body) que incluiría la uri, el payload y el user-agent. Estos elementos están en discusión y la definición requiere un análisis más profundo sobre el dataset en particular. Para la ingeniería de características se podrían hacer varios cálculos sobre la línea en sí, incluyendo: entropía de Shannon, puerto destino, cantidad de caracteres especiales, cantidad de palabras clave, ratio de caracteres especiales sobre el total, largo de la línea, etc.

d) Extracción de embeddings

Sobre las variables seleccionadas y concatenadas, debería realizarse la extracción de embeddings, a partir de distintas librerías de transformers para probar cuál aporta mejores resultados. En este punto es necesario tomar otra decisión metodológica, relacionada al largo de la línea a tomar: si truncar los resultados sobre el largo de la mayoría y perder parte de los datos (usando un percentil de largo de 95 por ejemplo) o utilizar el máximo perjudicando la velocidad y el consumo de recursos. Entre las librerías que se podrían considerar se encuentran: SecBERT y SecRoBERTa [16], DeBERTa [17], RoBERTa [18], SecureBERT [19], CANINE [20] y DistilRoBERTa [21], aunque la idea no es probar todas. En este punto no tendría sentido mantener los datos en el motor SQL; debería exportarse a un archivo que puedan consumir los algoritmos de entrenamiento como podría ser parquet o archivos de matrices.

e) Entrenamiento

Con los datos exportados, probar el entrenamiento con distintos algoritmos de clasificación: variantes de *gradient boosting* —LightGBM [22], XGBoost [23] y CatBoost [24]—, Support Vector Machine [25] y PyTorch [26], variando hiperparámetros, porcentajes de entrenamiento y pruebas sobre el dataset.

f) Prueba, pruebas cruzadas y recolección de métricas

El modelo entrenado, además de probarse con los datos que se apartaron para pruebas del mismo dataset, podrían testearse de forma cruzada con datos de otro dataset para ver la flexibilidad del modelo.

7.1. Técnicas / Herramientas

A continuación se detallan varias de las tecnologías, librerías y algoritmos concretos que se emplearán para ejecutar cada una de las etapas descritas en el punto 7. Es probable que en la etapa de experimentación algunas se reemplacen o se añadan otras.

- **Infraestructura:** Kubernetes como orquestador de contenedores para las distintas etapas del pipeline, y PostgreSQL como motor de base de datos para la ingesta inicial de los logs. Para orquestar y ejecutar las pruebas cross entre transformers y clasificadores, Apache Airflow conectado a Kubernetes con KPO (Kubernetes Pod Operator) para que cada tarea construya su propio contenedor aislado y efímero.
- **Procesamiento y limpieza de datos:** Python como lenguaje principal, junto con las librerías pandas y NumPy para la manipulación tabular, el tratamiento de valores nulos y la concatenación de campos.
- **Extracción de embeddings:** HuggingFace Transformers (`AutoTokenizer`, `AutoModel`) sobre modelos preentrenados de dominio como SecBERT [16] entre otros, con PyTorch [26] como backend de inferencia (con soporte GPU/CPU). La tokenización se realiza con truncamiento y padding a un largo fijo (aún no determinado).
- **Manejo de datos a gran escala:** Como el entorno de desarrollo está pensado para una laptop personal (con GPU incorporada) es necesario realizar checkpointing de los embeddings por lote en el disco (formato `.npy`) para tolerar mejor los fallos y reanudar procesos que mata el Sistema Operativo. La escritura en formato Parquet (vía PyArrow) para evitar la acumulación de metadata en memoria y si es necesario, en forma de streaming o chunks pequeños.
- **Entrenamiento y clasificación:** scikit-learn para la partición de datos (`train-test-split`), la binarización de etiquetas multi-clase y el cálculo de métricas; LightGBM como algoritmo de clasificación, además de los otros mencionados.
- **Evaluación:** Métricas de precisión, recall, f1-score y soporte por clase, junto con promedios micro, macro, ponderado y por muestra, estos últimos particularmente relevantes dado el fuerte desbalance entre clases del dataset.

8. Referencias-Bibliografía

[1] IBM Security, «Cost of a Data Breach Report 2025», IBM, Report, 2025, Disponible bajo registro. visitado 18 de jul. de 2026. dirección: `https://www.ibm.com/reports/data-breach`

[2] Akamai, *CVE-2021-44228 – Zero Day Vulnerability in Apache Log4j That Allows Remote Code Execution (RCE)*, Akamai Blog, 11 dic. 2021. https://www.akamai.com/blog/news/cve-2021-44228-zero-day-vulnerability, Accedido: 18-jul-2026, 2021.

[3] National Institute of Standards and Technology (NIST) – National Vulnerability Database, *CVE-2021-44228 Detail*, https://nvd.nist.gov/vuln/detail/CVE-2021-44228, Accedido: 18-jul-2026, 2021.

[4] R. Hiesgen, M. Nawrocki, T. C. Schmidt y M. Wählisch, «The Race to the Vulnerable: Measuring the Log4j Shell Incident», en *Proceedings of the Network Traffic Measurement and Analysis Conference (TMA '22)*, IFIP, 2022, págs. 1-9. dirección: https://arxiv.org/abs/2205.02544

[5] A. Muñoz y O. Mirosh, *A Journey from JNDI/LDAP Manipulation to Remote Code Execution Dream Land*, Black Hat USA 2016. https://www.blackhat.com/docs/us-16/materials/us-16-Munoz-A-Journey-From-JNDI-LDAP-Manipulation-To-RCE-wp.pdf, Accedido: 18-jul-2026, 2016.

[6] Apache Software Foundation – GitHub Repository, *apache/logging-log4j2: MessagePatternConverter.java (Historial de commits)*, https://github.com/apache/logging-log4j2/commits/be881e503e14b267fb8a8f94b6d15eddba7ed8c4/log4j-core/src/main/java/org/apache/logging/log4j/core/pattern/MessagePatternConverter.java, Accedido: 18-jul-2026, 2013.

[7] National Institute of Standards and Technology (NIST) – National Vulnerability Database, *CVE-2021-44832 Detail*, https://nvd.nist.gov/vuln/detail/CVE-2021-44832, Accedido: 18-jul-2026, 2021.

[8] J. Doll, C. McCarthy, H. McDougall y S. Bhunia, «Unraveling Log4Shell: Analyzing the Impact and Response to the Log4j Vulnerability», *arXiv preprint arXiv:2501.17760*, 2025. dirección: https://arxiv.org/abs/2501.17760

[9] T. Sureda Riera, J.-R. Bermejo-Higuera, J. Bermejo-Higuera, J.-J. Martínez-Herráiz y J.-A. Sicilia-Montalvo, «A New Multi-Label Dataset for Web Attacks CAPEC Classification Using Machine Learning Techniques», *Computers & Security*, vol. 120, pág. 102 788, 2022. DOI: 10.1016/j.cose.2022.102788

[10] A. F. Hassan, «Improving Web Application Security via Firewall-based detection and prevention of SQL injection attacks», *Journal of Digital Security and Forensics*, vol. 3, n.º 1, págs. 67-74, 2026. dirección: https://doi.org/10.29121/digisecforensics.v3.i1.2026.104

[11] N. Montes, G. Betarte, R. Martínez y A. Pardo, «Web application attacks detection using deep learning», en *Iberoamerican Congress on Pattern Recognition*, Springer, 2021, págs. 227-236. dirección: https://link.springer.com/chapter/10.1007/978-3-030-93420-0_22

[12] U. Aharon, R. Marbel, R. Dubin, A. Dvir y C. Hajaj, «RoBERTa-Augmented Synthesis for Detecting Malicious API Requests», *arXiv preprint arXiv:2405.11258*, 2024. dirección: https://arxiv.org/abs/2405.11258

[13] H. Karacan y M. Sevri, «A novel data augmentation technique and deep learning model for web application security», *IEEE Access*, vol. 9, págs. 150 781-150 797, 2021. dirección: https://www.academia.edu/108290354/A_Novel_Data_Augmentation_Technique_and_Deep_Learning_Model_for_Web_Application_Security

[14] T. Chen et al., «A payload based malicious http traffic detection method using transfer semi-supervised learning», *Applied Sciences*, vol. 11, n.º 16, pág. 7188, 2021. dirección: https://doi.org/10.3390/app11167188

[15] J. D. Day y H. Zimmermann, «The OSI reference model», *Proceedings of the IEEE*, vol. 71, n.º 12, págs. 1334-1340, 1983. dirección: https://ieeexplore.ieee.org/abstract/document/1457043

[16] Jackaduma, *SecBERT: A Pre-trained Language Model for Cybersecurity*, Accedido: 2026, 2022. dirección: https://huggingface.co/jackaduma/SecBERT

[17] P. He, X. Liu, J. Gao y W. Chen, «DeBERTa: Decoding-enhanced BERT with Disentangled Attention», en *International Conference on Learning Representations (ICLR)*, 2021. arXiv: 2006.03654. dirección: https://openreview.net/forum?id=XPZIaotutsD

[18] Y. Liu et al., «RoBERTa: A Robustly Optimized BERT Pretraining Approach», *arXiv preprint arXiv:1907.11692*, 2019. arXiv: 1907.11692. dirección: https://arxiv.org/abs/1907.11692

[19] E. Aghaei, X. Niu, W. Shadid y E. Al-Shaer, «SecureBERT: A Domain-Specific Language Model for Cybersecurity», en *Security and Privacy in Communication Networks (SecureComm 2022)*, ép. LNICST, vol. 462, Springer, 2023, págs. 39-56. DOI: 10.1007/978-3-031-25538-0_3 dirección: https://doi.org/10.1007/978-3-031-25538-0_3

[20] J. H. Clark, D. Garrette, I. Turc y J. Wieting, «CANINE: Pre-training an Efficient Tokenization-Free Encoder for Language Representation», *Transactions of the Association for Computational Linguistics*, vol. 10, págs. 73-91, 2022. DOI: 10.1162/tacl_a_00448 dirección: https://doi.org/10.1162/tacl_a_00448

[21] V. Sanh, L. Debut, J. Chaumond y T. Wolf, «DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter», *arXiv preprint arXiv:1910.01108*, 2019. arXiv: 1910.01108. dirección: https://arxiv.org/abs/1910.01108

[22] G. Ke et al., «LightGBM: A Highly Efficient Gradient Boosting Decision Tree», en *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, págs. 3146-3154. dirección: https://proceedings.neurips.cc/paper_files/paper/2017/file/6449f44a102fde848669bdd9eb6b76fa-Paper.pdf

[23] T. Chen y C. Guestrin, «XGBoost: A Scalable Tree Boosting System», en *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, págs. 785-794. DOI: 10.1145/2939672.2939785 dirección: https://doi.org/10.1145/2939672.2939785

[24] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush y A. Gulin, «CatBoost: Unbiased Boosting with Categorical Features», en *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 31, 2018. arXiv: 1706.09516. dirección: https://arxiv.org/abs/1706.09516

[25] C. Cortes y V. Vapnik, «Support-vector Networks», *Machine Learning*, vol. 20, n.º 3, págs. 273-297, 1995. DOI: 10.1007/BF00994018 dirección: https://doi.org/10.1007/BF00994018

[26] A. Paszke et al., «PyTorch: An Imperative Style, High-Performance Deep Learning Library», en *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019, págs. 8024-8035. arXiv: 1912.01703. dirección: https://arxiv.org/abs/1912.01703